Section A - Data Engineering Theory

Q1. OLTP vs OLAP Architecture

The company currently stores transactional and analytical workloads in the same PostgreSQL database.

Tasks

1. Explain why this creates performance issues.

   OLTP (Online Transaction Processing) and OLAP (Online Analytical Processing) have opposite characteristics. OLTP manage day-to-day operational transactions with very simple operations (insert, update, delete) and queries. Whereas OLAP supports analytics with complex queries and aggregations requiring large scans that could delay, block or time-out OLTP writes (customer transactions).

2. Compare OLTP and OLAP workloads.
   The data volume managed by OLTP is current operational data whilst OLAP manages historical large datasets. OLTP needs fast writes whereas OLAP fast reads since OLAP's query complexity is very simple and OLAP's is complex, having to scan millions of rows per query. The storage is designed row-oriented for OLTP and column-oriented for OLAP.

3. Suggest an improved architecture.

   Medallion architecture or modern data lakehouse. Extracting data in batches from PostgreSQL (OLTP) using airflow, storing raw data into a Data lake (bronze layer), storing transformations with cleaned and validated data in the silver layer of the data lake, storing aggregated and business ready tables in the data warehouse (gold layer). End users (Dashboards and ML) query only the gold layer with no impact in OLTP operations.

4. Explain where: -PostgreSQL /SQLServer -Data warehouse -Data lake
   should be used.

   PostgreSQL: Used as the as the operational transactional database (OLTP). It handles real-time orders and user profiles.
   Data Lake: Used for Storage. It holds massive volumes of raw, unstructured clickstream logs cheaply.
   Data Warehouse: Used for Analytics. It holds cleaned, structured data optimized for dashboards and SQL querying.

Q2. ACID vs BASE in Distributed Systems

QuickCart wants to store clickstream events from millions of users.

Tasks

1. Explain why BASE systems are preferred for clickstream systems.

Clickstream events involve millions of events. The priority is not immediate consistency but high availability. If one click event is recorded a few seconds later in reports, business is not affected.

2. Which workloads require ACID compliance?

   Financial transactions and Inventory management require ACID, consistency is key and not negotiable. Transactions must be 100% accurate.

3. Give examples where eventual consistency is acceptable.

   Social media likes or product recommendations, since real time precision is unnecessary.

Q3. Storage Formats & Querying

Tasks

1. Compare: -CSV -JSON -Avro -Parquet

   CSV/JSON: text / semi-structured text row-based, human-readable, universal. Flexible in structure. Large file sizes (no compression) slow analytical queries.

   Avro: binary row-based, moderate performance for analytical queries and medium storage size. Excellent for write-heavy systems and schema evolution.

   Parquet: columnar storage, excellent storage size, optimized for analytical read-heavy workloads.

2. Why do analytical systems prefer columnar storage?

   It improves the computing by reading only the required columns instead of scanning the whole table. It also reduces the storage due to the high compression

3. Explain predicate pushdown.

   This is an optimization where the filtering logic (WHERE city = 'Madrid') is "pushed" down to the storage layer. Instead of reading all rows and filtering later, Parquet allows engines to skip reading irrelevant data.

4. Explain compression benefits in Parquet.

   Parquet uses built-in compression, similar column values are stored together. This reduces drastically the storage size and consequently storage costs. It also allows faster queries by optimizing resources.

Q4. Partitioning, Indexing & Sharding

Tasks

1. Design a partitioning strategy for: -orders table -event data

Orders table partitioned order date (year and month). Event data partitioned by event date (year, month and day).

2. Suggest indexing strategy for: -customer lookups -order filtering -joins

   Customer Lookups: index on customer_id.
   Order Filtering: index order_time.
   Joins: Foreign Keys (product_id, customer_id) are indexed.

3. Explain when indexes negatively affect performance.

   Every index adds overhead to Write operations (INSERT/UPDATE/DELETE). The database must update the index every time data changes. Too many indexes will slow down the order processing.

4. Explain horizontal sharding.

   Sharding (also called horizontal partitioning) is the practice of splitting a large dataset table into smaller, independent pieces called shards in different nodes. It has high scalability as we can add as many nodes as we want. It is used to balance the data and the writes (traffic). Challenges: avoid hot shards and cross-shard queries, joins, aggregations.